



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

## Lab Experiment – 09

### Travelling Salesman Problem (TSP)

#### Aim

To implement the Travelling Salesman Problem (TSP) using Python.

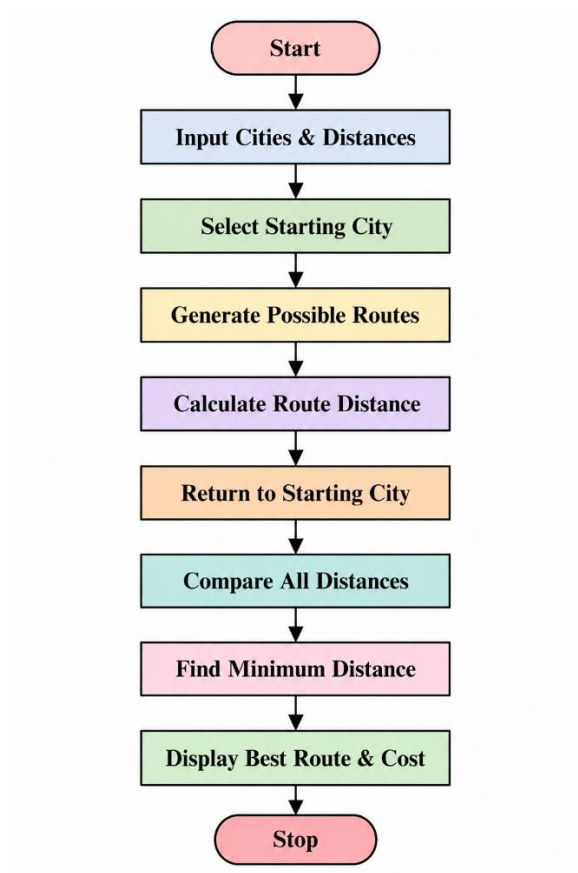
#### Objective

To find the minimum-cost route that visits every city exactly once and returns to the starting city.

#### Algorithm

1. Start with a set of cities and their distances.
2. Select a starting city.
3. Generate all possible routes through the remaining cities.
4. Calculate the total distance of each route.
5. Add the distance from the last city back to the starting city.
6. Compare the total distances.
7. Select the route having the minimum distance.
8. Display the shortest route and its total cost.
9. Stop.

## FlowChart



## Code

```
from itertools import permutations
```

```
cities = ['A', 'B', 'C', 'D']
```

```
distance = {  
    'A': {'A': 0, 'B': 10, 'C': 15, 'D': 20},  
    'B': {'A': 10, 'B': 0, 'C': 35, 'D': 25},  
    'C': {'A': 15, 'B': 35, 'C': 0, 'D': 30},  
    'D': {'A': 20, 'B': 25, 'C': 30, 'D': 0}  
}
```

```
start = 'A'
```

```
remaining = [city for city in cities if city != start]
```

```
min_distance = float('inf')
```

```
best_route = None
```

```
for route in permutations(remaining):
```

```
    current_route = (start,) + route + (start,)
```

```
    total_distance = 0
```

```
    for i in range(len(current_route) - 1):
```

```
        total_distance += distance[current_route[i]][current_route[i + 1]]
```

```
    if total_distance < min_distance:
```

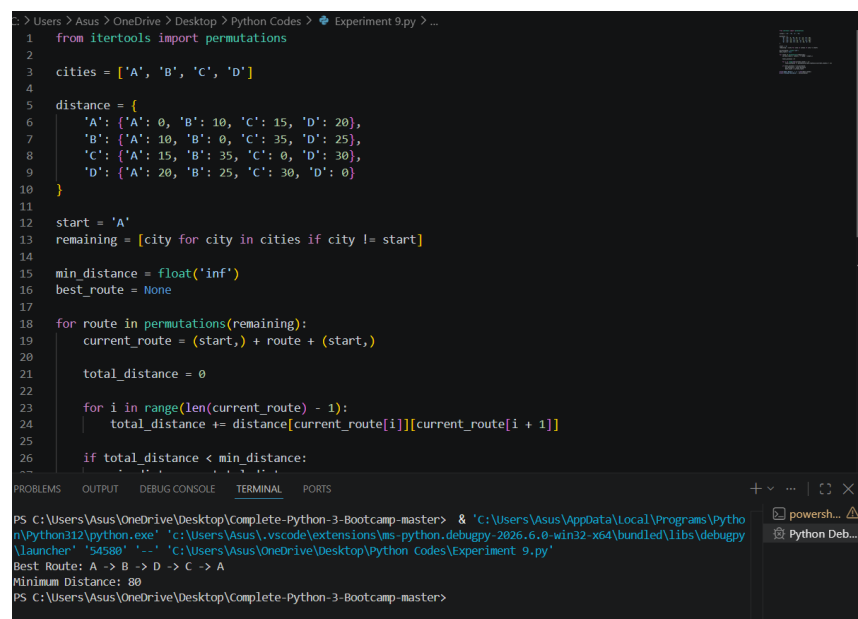
```
        min_distance = total_distance
```

```
        best_route = current_route
```

```
print("Best Route:", " -> ".join(best_route))
```

```
print("Minimum Distance:", min_distance)
```

## Output



```
1 from itertools import permutations
2
3 cities = ['A', 'B', 'C', 'D']
4
5 distance = {
6     'A': {'A': 0, 'B': 10, 'C': 15, 'D': 20},
7     'B': {'A': 10, 'B': 0, 'C': 35, 'D': 25},
8     'C': {'A': 15, 'B': 35, 'C': 0, 'D': 30},
9     'D': {'A': 20, 'B': 25, 'C': 30, 'D': 0}
10 }
11
12 start = 'A'
13 remaining = [city for city in cities if city != start]
14
15 min_distance = float('inf')
16 best_route = None
17
18 for route in permutations(remaining):
19     current_route = (start,) + route + (start,)
20
21     total_distance = 0
22
23     for i in range(len(current_route) - 1):
24         total_distance += distance[current_route[i]][current_route[i + 1]]
25
26     if total_distance < min_distance:
```

Best Route: A -> B -> D -> C -> A  
Minimum Distance: 80

## **Result**

The Travelling Salesman Problem was successfully implemented using Python, and the minimum-cost route was obtained.

## **Conclusion**

The program examines the possible routes and selects the route with the minimum total travelling distance while visiting each city exactly once and returning to the starting city.